



REACT 19 + VITE 8

EXPRESS 5 + NODE.JS

MONGODB ATLAS 9

PROGRESSIVE WEB APP (PWA)

WEB AUDIO API

OXLINT VERIFIED

Comprehensive Technical Specifications & Architecture Whitepaper

A mood-aware, state-driven action planner and distraction-free execution engine. Eliminates decision paralysis through real-time physiological & emotional vector scoring, whole-app focus locking, and micro-learning downtime feeds.

"Your state changes. Your next move shifts. No endless to-do lists. No decision fatigue. Just the single thing worth doing right now."

APPLICATION NAME

SHIFT (shift-momentum-on-demand)

LEAD ARCHITECT & DEVELOPER

Dhruv Solanki

PRIMARY LANGUAGES

JavaScript (ES2024), JSX, CSS3, HTML5, SVG

APPLICATION TYPE

Mobile-First Progressive Web App (PWA)

DEVELOPMENT TOOLING

Visual Studio Code & Antigravity IDE

SPECIFICATION VERSION & DATE

Release v1.0.0 — October 2026

SHIFT is a purpose-built Progressive Web Application designed to solve the endemic failure mode of traditional task managers: *choice overload and context blindness*. Unlike legacy productivity software (Todoist, TickTick, Notion, Asana) which function as passive, guilt-inducing backlogs of uncompleted obligations, SHIFT acts as an active, state-aware recommendation engine. It takes three real-time human inputs (Emotional Headspace, Physiological Energy Tank, and Available Temporal Window), filters an active task pool, and returns a single, unambiguous next move.

Document Structure & Chapter Navigation

1. The Modern Productivity Dilemma	Problems	2. How SHIFT Is The Solution	Philosophy
3. Comprehensive Feature Suite	Core System	4. Unique Selling Points (USPs)	Differentiators
5. Development Environment & Languages	Dev Stack	6. Progressive Web App (PWA) Architecture	Client Model
7. Complete Library & Framework Registry	Dependencies	8. Backend Architecture & REST Schemas	REST / DB
9. Recommendation Engine Mathematics	Algorithms	10. Downtime Engine & External APIs	Micro-Learning
11. Zero-Dependency Data Visualizations	SVG Charts	12. Hardware, Audio & Particle Physics	Web APIs
13. Performance, Security & Roadmap	Production		

1 Problems We Face Nowadays (The Productivity Dilemma)

Despite hundreds of task management tools available today, modern knowledge workers, students, and engineers suffer from heightened chronic burnout and severe procrastination. An analysis of modern productivity dynamics reveals four fatal structural failures:

 **1. Backlog Paralysis & Guilt Debt**

Standard to-do lists act as dumping grounds. Users pile up 50 to 200 tasks. Opening the app creates instant decision fatigue (Hick's Law). The cognitive overhead of choosing what to do paralyzes the user, while overdue red badges induce guilt, anxiety, and eventual app abandonment.

 **2. Cognitive State & Energy Disconnect**

Traditional apps treat humans as deterministic CPUs with uniform operational capacity. They ignore emotional fatigue, post-lunch energy crashes, or mental burnout. Recommending high-intensity analytical work to a depleted brain guarantees failure, frustration, and avoidance loops.

 **3. Fragmented Time & Doomscrolling Trap**

Modern work is fragmented into micro-windows of 5 to 15 minutes between meetings or chores. Because large projects cannot be completed in 10 minutes, users reflexively default to passive doomscrolling on social media (reels, feeds), depleting mental energy without restorative rest.

 **4. "Productivity Theater" Syndrome**

Users waste hours categorizing, color-coding, creating nested sub-folders, tagging, and rearranging Kanban boards. This gives a false dopamine rush of accomplishment without moving the needle on actual execution, turning organization into a sophisticated form of procrastination.

SHIFT fundamentally reverses the relationship between the human and the backlog. Instead of forcing you to organize the list, SHIFT hides the backlog and hands you **exactly one move** tailored to your immediate biological and emotional state:

The "One Move" Paradigm

Zero list browsing at the moment of execution. SHIFT assesses your state, queries your tasks, runs its scoring engine, and presents a single, unmistakable action card. Decision fatigue drops to zero, restoring clean forward momentum.

Bi-Directional Effort ↔ Energy Fit

SHIFT completely decouples duration from cognitive difficulty. A 10-minute task can be exhausting (filing taxes), while a 45-minute task can be relaxing (cleaning a desk). SHIFT matches the gas in your tank to the effort the task demands.

The Focus Lock Sanctuary

When an action begins, the entire application interface locks. Navigation disappears. The user cannot browse tasks, history, or insights. A clean countdown ring keeps the user honest, mirrored across all active devices in real-time.

Constructive Downtime Feeds

For micro-breaks, SHIFT provides a built-in algorithmic antidote to doomscrolling: Word Forge (lexical mastery) and Deep Read (curated Wikipedia summaries), with automated 2.5s dwell-detection logging.

SHIFT End-to-End System Execution Loop

[Check-In State: Mood · Energy · Time Window] → [Candidate Task Pool Filter (Hard Duration Ceiling)]
→ [Scoring Matrix: Effort Gap ± Mood Resonance ± Priority Hierarchy ± Category Novelty ± Feedback Bias]
→ [Single Highest-Scoring Recommendation Card Presented] → [Slide-to-Confirm Physics Drag]
→ [App-Wide Focus Lock Engaged (Active Sync across all client devices)] → [SVG Ring Countdown & Overtime]
→ [Complete: Web Audio Harmonic Triad (C5-E5-G5) + Haptics] → [Feedback Loop (+25/-25 Reinforcement)]

The Psychological Breakthrough: Momentum Over Planning

Traditional planning tools demand high executive function at the exact moments when willpower is depleted. SHIFT lowers the barrier to entry to an absolute minimum: answer three quick chips, slide the bar, and execute one single move. Momentum builds confidence, and confidence breaks procrastination cycles.

The application is structured into four main operational views, an ambient discovery feed, and an immersive focus execution state:

Module / View	User Workflow & Interactions	Underlying System Behavior	Key Components
Now (Check-In)	User answers 3 questions: Mood (6 options), Energy (3 options), Available Time (4 options). Taps "SHIFT →".	Stores session in MongoDB, executes scoring pipeline, evaluates hard time boundaries, renders single recommendation card.	Now.jsx, FlipText.jsx, SlideToConfirm.jsx
Focus Lock	Triggered by sliding the confirmation handle. Locks out all navigation and page mounts across the client.	Initializes countdown ring against server startedAt. Transitions seamlessly to overtime counting. Polls active action sync.	FocusLock.jsx, activeAction.js, Web Audio API
Tasks Pool	Create, update, soft-delete tasks. Filter by Active and Completed. Configure Effort, Priority, and Type.	Differentiates between "One-Off" and "Permanent" habit tasks. Enforces required fields. Performs optimistic updates.	Tasks.jsx, TaskFormFields.jsx, Modal.jsx
History & Log	Chronological calendar grouping all task completions and downtime reading sessions by local date.	Supports retroactive manual task logging with historical date picker and subjective post-task reflection notes.	History.jsx, resolve.js, dayKey()
Insights Engine	Visual analytics dashboard with 4 tabs: Overview, Rhythm, Streaks, and Trends.	Calculates completion rates, category streaks, 17-week GitHub-style heatmap, circadian 2D distribution, and mood correlation.	Insights.jsx, CountUp.jsx, Hand-rolled SVGs
Downtime Suite	Full-screen swipeable feed for micro-learning: Word Forge (lexicon) and Deep Read (Wikipedia digests).	Uses IntersectionObserver with 2.5s dwell threshold to automatically record completed sessions without buttons.	Downtime.jsx, DowntimeFeed.jsx, content.js

∞ Permanent vs One-Off Task Lifecycles

Tasks are split into two paradigms: oneoff tasks (e.g. "Fix broken door hinge") vanish from the pool upon completion, while permanent habits (e.g. "15-min core mobility") stay active indefinitely, tallying cumulative completion streaks and remaining available for future recommendation cycles.

🔄 Persistent Session State (flowCache)

The check-in selections and recommendation card survive tab switches without re-fetching via module-scoped flowCache. If a user switches to Tasks to check a detail, returning to Now preserves the exact recommendation until explicitly reset or started.

SHIFT incorporates eight distinct architectural innovations that do not exist in mainstream productivity platforms:

USP 01 — THE ANTI-LIST PARADIGM

Zero-Backlog Execution

Users never browse a list when it is time to work. The interface eliminates choice overload by reducing the decision space from N items to 1 mathematically optimized action card.

USP 03 — UNBREAKABLE SANCTUARY

Full-App Focus Lockout

Starting a task freezes the entire application shell. Route navigation, task pools, and history are completely unmounted until the user marks the move complete or skipped.

USP 05 — DELIBERATE INTENT

Slide-to-Confirm Physics

Replaces accidental taps with a 90% travel slide mechanic built with Web Animations API and cubic-bezier spring physics, enforcing conscious, mindful commitment.

USP 07 — INTENTIONAL REST

Dwell-Tracked Downtime Feed

Replaces algorithmic doomscrolling with curated Datamuse vocabulary and Wikipedia digests. Automatically records educational credit after 2.5s center-screen dwell.

USP 02 — COGNITIVE DECOUPLING

Effort ↔ Energy Matching

Decouples clock duration from cognitive load. A 15-minute high-focus task is penalized when low on energy, while a 45-minute low-effort task is rewarded, matching reality.

USP 04 — OMNIPRESENT COHESION

Cross-Device State Polling Sync

Starting a focus session on a laptop immediately locks the mobile PWA on your phone. Completing the task on one device instantly releases the lock everywhere.

USP 06 — ZERO-ASSET SYNTHESIS

Web Audio Harmonic Payoff

Synthesizes an uplifting C5-E5-G5 major triad sine chord via browser AudioContext and triggers physical haptic vibration. Zero MP3 assets; zero bundle overhead.

USP 08 — LEAN VECTOR VISUALS

100% Hand-Rolled SVG Visuals

Zero third-party charting libraries (no Chart.js, Recharts, or D3). All donuts, 17-week heatmaps, trend lines, and timers are pure, pixel-perfect, accessible SVGs.

The Unified Competitive Advantage

Unlike fragmented point solutions (e.g. Forest for timing, Notion for storage, Duolingo for breaks), SHIFT unifies task intake, state assessment, distraction-free execution, and restorative downtime into a single cohesive, ultra-lightweight Progressive Web App.

SHIFT is engineered as an ultra-fast, modern, decoupled full-stack architecture with strict separation of concerns, zero runtime CSS overhead, and lightweight vector graphics:

🔧 Engineering & Development Tools

- **Visual Studio Code:** Primary IDE with multi-root workspace support, Git integration, and ES Modules syntax support.
- **Antigravity IDE:** Advanced agentic AI pair programming environment, autonomous debugging, code inspection, and architectural verification.
- **Git & GitHub:** Distributed version control, branch isolation, and release tagging.
- **Node Package Manager (npm v10+):** Dependency resolution and build scripts.
- **Oxlint (Rust Linter):** Ultra-fast static analysis ensuring code hygiene.

📄 Languages & Technical Standards

- **JavaScript (ES2024):** ES Modules on frontend, CommonJS on server, async/await, optional chaining, nullish coalescing, AbortController.
- **JSX:** React 19 component declarative composition syntax.
- **Vanilla CSS3 (Design Tokens):** CSS Custom Properties, Flexbox, CSS Grid, Web Animations API, CSS @page print rules.
- **HTML5 & SVG:** Semantic elements (<article>, <section>, <nav>, <svg>).

Type of Application: Progressive Web App (PWA)

SHIFT is built from the ground up as a **Mobile-First Progressive Web Application**. It provides a native app experience on iOS, Android, and Desktop without app store friction:

- **Standalone Display Mode:** Configured with `display: "standalone"`, removing browser address bars and navigation controls for a native feel.
- **Offline Resilience & Auto-Update:** Managed by `vite-plugin-pwa` with `registerType: "autoUpdate"`, guaranteeing background service worker cache refreshes.
- **Theme Consistency:** Enforces `theme_color: "#0b0f14"` and `background_color: "#0b0f14"` to ensure seamless status bar and splash screen integration.
- **Maskable Adaptive Icons:** Full compliance with Android adaptive icons (192×192, 512×512, and maskable 512×512).
- **Fluid Gesture Shell:** Embla Carousel provides hardware-accelerated horizontal touch gestures across views and nested Insights sub-tabs.

```
// PWA Configuration (client/vite.config.js)
VitePWA({ registerType: 'autoUpdate', includeAssets: ['favicon.svg', 'apple-touch-icon.png'],
manifest: { name: 'SHIFT — Momentum on demand', display: 'standalone', theme_color: '#0b0f14' } })
```

Detailed inventory of all imported dependencies across frontend client and backend server layers:

Library / Package	Version	Layer	Role, Technical Description & Justification
react / react-dom	^19.2.8	Client UI	Core reactive component framework. Leverages React 19's optimized reconciler and compiler for sub-millisecond DOM updates.
vite	^8.3.0	Build Tool	Next-generation frontend tooling. Instant Hot Module Replacement (HMR) and optimized Rollup tree-shaking production builds.
react-router-dom	^7.18.4	Routing	Client-side declarative routing, URL state management, and seamless transitions between Desktop sidebar and Mobile shell.
embla-carousel-react	^8.6.0	Gestures	Ultra-lightweight, 60fps touch physics engine powering the mobile swipe pager and nested horizontal tab transitions.
vite-plugin-pwa	^1.3.0	PWA Runtime	Automates Workbox service worker generation, web manifest compilation, and asset caching strategies for offline support.
express	^5.2.1	Server API	Lightweight, unopinionated Node.js server framework managing REST routing, middleware chaining, and robust error dispatch.
mongoose	^9.10.1	Database ORM	MongoDB object modeling library providing schema validation, casting, atomic updates, and relational document population.
jsonwebtoken	^9.0.2	Security / Auth	Cryptographic signing and verification of stateless JSON Web Tokens (JWT) for secure, tamper-proof user sessions.
bcryptjs	^2.4.3	Security / Hash	Secure Blowfish password hashing with 10 salt rounds, protecting user credentials against rainbow tables and brute force.
cors	^2.8.6	Middleware	Cross-Origin Resource Sharing middleware enabling strict origin whitelisting in production environments.
dotenv	^17.4.2	Configuration	Loads sensitive environment variables (MONGO_URI, JWT_SECRET) into process.env securely.
oxlint	^1.81.0	Linter	Next-generation Rust-based linter that validates the client codebase in milliseconds, catching bugs before runtime.
sharp	^0.35.4	Image Pipeline	High-performance Libvips C-library wrapper used for converting and resizing PWA icons and splash screens.

Zero Dependency Bloat Policy

Noticeable omissions are deliberate: **No CSS frameworks** (Tailwind, Bootstrap), **no animation bloat** (Framer Motion at 120KB), and **no charting libraries** (Chart.js / D3 at 200KB+). The entire client bundle transfers in under 180KB gzip, resulting in instantaneous cold starts even on 3G cellular connections.

The backend follows a defensive, multi-layered REST architectural pattern. All endpoints (except `/api/auth/*`) are strictly guarded by `authMiddleware`, which extracts and verifies the Bearer JWT, injecting `req.userId` to enforce rigid single-tenant database scoping.

Complete REST API Endpoint Directory

Method	Endpoint	Auth Guard	Purpose & Data Payload
POST	<code>/api/auth/register</code>	Public	Creates a new user profile with bcrypt salted password hash. Returns JWT.
POST	<code>/api/auth/login</code>	Public	Validates email/password credentials; issues signed JWT bearer token.
GET	<code>/api/auth/me</code>	Bearer JWT	Returns authenticated user record (id, name, email) for session hydration.
GET · POST	<code>/api/tasks</code>	Bearer JWT	Lists all active tasks or creates a new task (title, category, effort, time, type).
PUT · DEL	<code>/api/tasks/:id</code>	Bearer JWT	Updates task properties or performs soft deletion (status: 'deleted').
POST	<code>/api/sessions</code>	Bearer JWT	Logs a real-time check-in record (mood, energy, availableTime).
POST	<code>/api/recommendation</code>	Bearer JWT	Executes scoring engine against session ID; returns single optimal task.
GET · POST	<code>/api/actions</code>	Bearer JWT	Fetches user action history or initiates an active focus session.
PUT	<code>/api/actions/:id</code>	Bearer JWT	Updates action status (completed/skipped) and attaches feedback/notes.
GET · POST	<code>/api/activity</code>	Bearer JWT	Fetches downtime totals or logs completed reading / vocabulary sessions.

Database Collections & Mongoose Schemas (MongoDB Atlas)

 **User (User.js)** &  **Task (Task.js)**

User: name, email (unique index), password (bcrypt hash).

Task: user (ref), title, category, estimatedTime, priority (low/med/high), effort (low/med/high), type (oneoff/permanent), status (active/completed/deleted), completionCount.

 **Action (Action.js)** &  **Activity (Activity.js)**

Action: user (ref), sessionId, taskId (ref), source (shift/manual), status (started/completed/skipped), feedback (better/same/worse), note, startedAt, completedAt.

Activity: user (ref), type (read/vocab), title, completedAt.

The recommendation algorithm lives in `server/services/recommendationEngine.js`. It is a deterministic, test-covered scoring engine. It avoids black-box machine learning to guarantee 100% explainability, zero latency, and zero cloud API dependency.

THE SHIFT SCORING FORMULA

FinalScore(T) = BaseScore(50) + Δ_EnergyEffort + Δ_Mood + Δ_Priority + Δ_Variety + Σ(Δ_Feedback)

Evaluation Vector	Condition / Mathematical Formula	Score Delta	Behavioral Rationale
Temporal Hard Limit	<code>estimatedTime > availableTime</code>	DISQUALIFIED	Strict ceiling. A 30-min task is never returned when the user only has 15 minutes. Returns 404 (no-fit) if no tasks qualify.
Energy ↔ Effort Match	<code>gap = effortLevel - energyLevel == 0</code>	+30	Dead-on match. The task demands exactly the cognitive reserve the user possesses.
Energy Surplus	<code>gap = effortLevel - energyLevel < 0</code>	+15	Comfortably doable. The user has high energy and the task is medium or low effort.
Cognitive Stretch	<code>gap == 1</code> (e.g., med effort, low energy)	-15	Slight stretch. Tolerable if priority or feedback compensates.
Cognitive Wall	<code>gap ≥ 2</code> (high effort, low energy)	-35	Severe cognitive wall. Prevents assigning complex analysis when drained.
Rough Headspace Filter	<code>moodScore ≤ 2 && effort == 3</code>	-15	When feeling "low" or "overwhelmed", tolerance for high-effort work collapses.
Positive Mood Resonance	<code>moodScore ≥ 3</code> ("good", "great")	+5	Slight positive resilience nudge across all candidate tasks.
Priority Hierarchy	<code>priority == "high" / "medium"</code>	+20 / +10	High-priority tasks receive systematic preference, balanced against energy.
Category Repetition Fatigue	<code>sameCategoryCount × 20</code> in recent actions	-20 / each	Prevents category monotony (e.g., doing 3 consecutive coding tasks).
Category Novelty Bonus	<code>sameCategoryCount == 0</code>	+10	Encourages healthy contextual cross-training across life domains.
Feedback Reinforcement	Historical "better" / "same" / "worse" / "skipped"	+25 / +5 / -25 / -10	Learns which specific tasks actually restore or deplete the user.

🧪 Verified Test Case: Energy Decoupling

In `recommendationEngine.test.js`: When a user checks in with **Low Energy** and 60 minutes available, the engine evaluates a 45-minute Low-Effort task vs. a 10-minute High-Effort task. The 45-minute task scores $50 + 30$ (match) = 80, while the 10-minute task scores $50 - 35$ (wall) = 15. The engine correctly selects the longer, easier task.

🔄 Verified Test Case: Feedback Adaptation

If a task was previously marked "better", it earns +25 points. If a user previously marked a task "worse", it loses -25 points. This adaptive weighting ensures the app learns the user's subjective responses over time without requiring cloud neural nets.

The Downtime Engine (`client/src/downtime/content.js` & `DowntimeFeed.jsx`) provides a clean, vertical snap feed that replaces brain-draining social media scrolling with structured micro-learning:

1. Word Forge (Lexicon Builder)

- **Query Engine:** Generates randomized letter patterns (4–9 chars) and queries the **Datamuse API** (`api.datamuse.com/words?sp=????&md=dp`).
- **POS Normalizer:** Maps part-of-speech codes (noun, verb, adjective, adverb) and extracts definitions.
- **Real Sentence Examples:** Asynchronously queries **DictionaryAPI.dev** (`api.dictionaryapi.dev/api/v2/entries/en/{word}`) with an `AbortController` to layer real-world context sentences into the card.

2. Deep Read (Wikipedia Engine)

- **Curated Clusters:** Three balanced knowledge clusters: Defence & Military History, Computer Science, and Future Tech / Frontier Science.
- **MediaWiki Query API:** Queries `en.wikipedia.org/w/api.php?action=query&prop=extracts|pageimages|description&explaintext=1`.
- **Structured Block Parser:** Strips trailing citation links and parses wikitext headings (`= Heading =`) into structured typography blocks with thumbnail images.

Automated Dwell-Tracking Mechanism (Zero-Click Activity Logging)

Instead of requiring the user to tap an artificial "Done" button, `DowntimeFeed.jsx` deploys a native `IntersectionObserver` with a thin central threshold band. When a card crosses the vertical center of the screen and remains visible for more than **2,500ms (2.5 seconds)**, a background activity log is dispatched to `/api/activity`, recording the learning milestone automatically.

```
// IntersectionObserver Center-Band Dwell Trigger (DowntimeFeed.jsx)
const io = new IntersectionObserver((entries) => {
  entries.forEach(e => {
    if (e.isIntersecting) {
      timers.set(key, setTimeout(() => logActivity(logKind, title), 2500));
    } else { clearTimeout(timers.get(key)); timers.delete(key); }
  });
}, { rootMargin: "-45% 0px -45% 0px" }); // Thin 10% center detection band
```

Resilient Network Fetching with AbortController

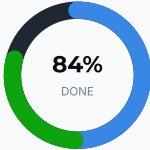
All external API calls (Datamuse, DictionaryAPI, Wikipedia) feature strict timeout abort controllers (capped at 4s for dictionary and 10s for Wikipedia) and automatic unmount teardown, ensuring that flaky mobile network connections fail fast rather than hanging the user interface.

Rather than importing bloated charting bundles (e.g., Chart.js at 150KB or Recharts at 400KB), SHIFT uses **100% hand-crafted SVG primitives** styled directly via CSS design tokens:

Visualization Primitive	Technical Implementation	Rendered Metrics & Insights
Donut Charts (Donut)	SVG circle stroke manipulation with strokeDasharray, strokeDashoffset, and 3px gap calculations.	Category breakdown, feedback sentiment split (better vs same vs worse), and task status distribution.
Activity Heatmap (Heatmap)	17-week CSS grid of 119 date-keyed cells with 5-tier intensity color levels (heat-0 to heat-4).	GitHub-style daily momentum visualization mapping completions over the preceding 4 months.
Area Line Chart (LineChart)	Mathematical point normalization: polyline path paired with an SVG polygon gradient area underlay.	8-week trailing rolling completion rate trends with dynamic gridlines at 0%, 50%, and 100%.
Magnitude Bar List (BarList)	Normalized percentage bar fill widths: $(item.value / scaleMax) * 100\%$.	Relative category rankings, permanent task habit completion tallies, and untouched task counts.
Circadian 2D Rhythm Matrix	Dual breakdown mapping actions against Weekdays (Sun-Sat) and 4 Diurnal Buckets (Morning, Afternoon, Evening, Night).	Identifies peak biological productivity windows and circadian rhythms to schedule high-effort tasks effectively.

Live Vector SVG Demonstrations (Native Application Primitives)

SVG Donut Primitive



17-Wk Heatmap Grid



8-Wk Trend Sparkline



SHIFT delivers tactile, visceral satisfaction through direct browser hardware APIs without loading heavy multimedia assets:

🎵 Web Audio Harmonic Triad

Located in `celebratePayoff()` in `FocusLock.jsx`. Instantiates a native `AudioContext` and synthesizes a rising major triad:

- **C5:** 523.25 Hz
- **E5:** 659.25 Hz
- **G5:** 783.99 Hz

Each note uses an `OscillatorNode` (sine wave) connected to a `GainNode` with linear attack and exponential decay (0.32s). Zero audio file requests.

📱 Navigator Haptic Feedback

Directly interfaces with mobile device vibration motors via `navigator.vibrate([0, 35, 45, 35])`. Generates a distinct double-tap tactile pulse upon task completion, reinforcing feelings of progress and psychological closure. Automatically disabled when unsupported or when reduced motion is preferred.

Composer-Only Particle Engine (TapDroplets.jsx)

Mounted globally at the application root, `TapDroplets.jsx` provides ambient physical feedback on every pointer-down event without affecting frame rates:

🌀 Parabolic Particle Arc Mathematics

Spawns 6 lightweight droplet elements per tap. Particles travel along a randomized radial vector between 22px and 58px over 430ms to 680ms, influenced by a 26px gravitational downward sag.

🚫 Zero Filter Re-Rasterization

Crucially, droplets operate strictly via CSS `transform` and `opacity`. CSS `filter: blur()` was eliminated because animated SVG filters force CPU re-rasterization on every frame, which previously crashed mobile Safari tabs.

Slide-to-Confirm Intentionality Mechanism (SlideToConfirm.jsx)

```
// Web Animations API Spring Overshoot (SlideToConfirm.jsx)
handleRef.current.animate([
  { transform: handleRef.current.style.transform },
  { transform: "translateX(0)" }
], { duration: 320, easing: "cubic-bezier(0.34, 1.3, 0.64, 1)", fill: "forwards" });
```

Accessible Design & Prefers-Reduced-Motion Compliance

All kinetic effects across SHIFT—including particle droplets, slide-to-confirm springs, flip text typography, and audio/haptic celebration chimes—query `window.matchMedia("(prefers-reduced-motion: reduce)")`. When active, durations snap to 0ms and audio/vibration calls no-op cleanly.

⚡ Performance & Bundle Optimization

- **Zero CSS-in-JS Overhead:** Pure native CSS custom properties compiled into a single cacheable stylesheet (index.css).
- **Strict Memory Management:** All timers, observers, and event listeners feature automatic teardown via React useEffect return functions.
- **Compositor-Only Animations:** No repaints during slider or droplet animations; runs at 60/120fps on mobile.

🔒 Security & Tenant Isolation

- **Strict JWT Scoping:** All queries are hard-scoped via { user: req.userId }, preventing horizontal privilege escalation.
- **Bcrypt Salt Hashing:** 10 rounds of Blowfish encryption for stored credentials.
- **Boot Guard:** Process halts immediately if MONGO_URI or JWT_SECRET are missing from the environment.

Future Architectural Roadmap

Roadmap Milestone	Target Technology	Architectural Impact & Description
Phase 2: Local-First Offline Sync	IndexedDB + CRDTs / RxDB	Enables full offline task creation and local scoring when disconnected, syncing back to MongoDB Atlas upon reconnection.
Phase 3: Biometric Sensor Integration	Web Bluetooth / Apple HealthKit	Automatically pulls physiological stress and recovery indicators (Heart Rate Variability, Sleep Score) to populate the "Energy Tank".
Phase 4: Agentic Task Breakdown	Gemini API / LLM Microservice	Allows users to dump large, intimidating goals into an AI decompose pipeline that fractures them into 5-to-15 minute actionable bites.
Phase 5: Calendar Two-Way Sync	Google Calendar & CalDAV	Automatically detects free temporal slots between calendar meetings to dynamically recommend appropriate tasks.

SHIFT ⚡ — MOMENTUM ON DEMAND

Architected and Engineered by Dhruv Solanki • Released Under ISC License

SPECIFICATION COMPLETE & VALIDATED

Antigravity IDE & Visual Studio Code Verification Pass